

Atividade Prática 01

August 8, 2026

Contexto: o projeto

Você vai construir um repositório chamado **fae-relatorio**, contendo um curto relatório em $\text{\LaTeX}$ (1–2 páginas) sobre um tópico à sua escolha relacionado à disciplina (ex: “O que é o Shell”, “Sistemas de controle de versão”, “Ferramentas do $\text{\LaTeX}$ em FAE” — pode ser um resumo do que foi visto nos slides desta semana), com pelo menos uma referência bibliográfica via BibTeX, e um **Makefile** que compila esse relatório automaticamente (sem precisar digitar **pdflatex** na mão, várias vezes).

1 Repositório no GitHub

1. Crie uma conta no GitHub, se ainda não tiver. Observe que a autenticação deve ser feita agora por **token** agora.
2. Crie um novo repositório **vazio** (sem README) chamado F056.
3. No seu terminal, crie uma pasta local e inicialize o Git:

```
mkdir F05 && cd F056
git init
git branch -M main
```

4. Crie um arquivo README.md com uma descrição de 2–3 linhas do projeto, e um arquivo .gitignore contendo pelo menos:

```
*.aux
*.log
*.bbl
*.blg
*.out
*.pdf
```

(Note: ignoramos o **.pdf** porque ele deve ser **gerado pelo Make**, não versionado — o repositório guarda o *fonte*, não o resultado compilado.)

5. Faça o primeiro commit e conecte ao repositório remoto:

```
git add .
git commit -m "Commit inicial: Exercicio 01 de F056"
git remote add origin <URL-do-seu-repositorio>
git push -u origin main
```

Checkpoint: seu repositório no GitHub deve mostrar o README.md e o .gitignore após o push.

2 Documento $\text{\LaTeX}$ com BibTeX

Crie os seguintes arquivos na raiz do projeto:

- `relatorio.tex` — o documento principal, com pelo menos:
 - Título, seu nome, e uma seção de introdução.
 - Uma lista (`itemize` ou `enumerate`) resumindo 3 conceitos vistos nos slides desta semana (ex: git, make, sed).
 - Pelo menos **uma citação** usando `\cite{...}`, referenciando uma entrada do seu arquivo `.bib` (como no exemplo do slide “ $\text{\LaTeX}$: BibTeX e feynMF”).
 - Um `\bibliographystyle{plain}` e `\bibliography{referencias}` no final.
- `referencias.bib` — pelo menos uma entrada `@Article` ou `@Misc`, no mesmo formato do slide (ex: um artigo real sobre Física de Altas Energias, ou mesmo uma referência a este próprio curso).

Faça o commit desses dois arquivos (**não** commite o `.pdf` gerado — ele deve estar no `.gitignore`). Dê `git push`.

Checkpoint: `git log -oneline` deve mostrar um commit específico para o `.tex` e o `.bib`.

3 Makefile

No slide “ $\text{\LaTeX}$: BibTeX e feynMF”, compilar um documento com bibliografia exige rodar `latex/pdflatex`, depois `bibtex`, e depois `latex/pdflatex` mais duas vezes (para resolver as referências cruzadas e a bibliografia). Fazer isso manualmente é chato e sujeito a erro — é exatamente para isso que serve o `make`.

Na raiz do projeto, crie um `Makefile` que:

1. Defina uma macro `DOC = relatorio` (nome do documento, sem extensão), para não repetir o nome em cada linha.
2. Tenha um target `all` (ou `$(DOC).pdf`) que rode, na ordem correta:

```
pdflatex $(DOC).tex
bibtex $(DOC)
pdflatex $(DOC).tex
pdflatex $(DOC).tex
```

3. Tenha um target `clean` que remova todos os arquivos auxiliares gerados (`.aux`, `.log`, `.bbl`, `.blg`, `.out`) — **use a função wildcard** para não precisar listar cada extensão manualmente arquivo por arquivo (como no slide “Make: Funções”):

```
AUX_FILES = $(wildcard *.aux *.log *.bbl *.blg *.out)
clean:
    rm -f $(AUX_FILES)
```

4. **Atenção ao detalhe do slide:** os comandos shell dentro de cada regra devem começar com **TAB**, nunca espaços — se você usar espaços, o `make` vai falhar com “missing separator”. Esse é o erro mais comum desta atividade.

Teste do zero:

```
make clean
make all
```

Um arquivo `relatorio.pdf` deve ser gerado, com a citação aparecendo corretamente no texto e a bibliografia no final.

Commit e push do Makefile funcionando.

Checkpoint: rodar `make clean && make all` a partir de um clone limpo do repositório deve gerar o PDF completo, sem erros do L^AT_EX/BibTeX e sem que você precise digitar nenhum comando manualmente.

4 Branch, edição e merge

1. Crie um branch novo:

```
git branch feature-secao-conclusao
git checkout feature-secao-conclusao
```

2. Nesse branch, adicione uma nova seção `\section{Conclusão}` ao final de `relatorio.tex`, com 2–3 frases. Faça commit.
3. Volte para o `main` e, **antes de mesclar**, altere a mesma região final do `relatorio.tex` (ex: mude o texto da última frase da introdução, ou adicione uma linha em branco/comentário logo antes de onde a nova seção entraria) e faça commit direto no `main`. Isso é proposital: você vai gerar um conflito real.

4. Tente mesclar:

```
git checkout main
git merge feature-secao-conclusao
```

5. O Git deve acusar um conflito em `relatorio.tex`. Abra o arquivo, resolva manualmente escolhendo (ou combinando) as duas versões, remova os marcadores `<<<<`, `=====`, `>>>>`, e finalize:

```
git add relatorio.tex
git commit
```

6. Rode `make clean && make all` de novo para confirmar que o documento ainda compila depois do merge. Dê `git push`.

Checkpoint: `git log -oneline -decorate -graph -all` deve mostrar visualmente o branch `feature-secao-conclusao` sendo mesclado ao `main`, incluindo o commit de merge.

5 Sed (peso 10%)

Usando **apenas um comando sed** (sem editar o arquivo manualmente), troque o estilo da bibliografia de `plain` para `unsrt` no seu `relatorio.tex`, no mesmo espírito do exemplo do slide (“Sed: Exemplo prático”):

```
sed -i 's/\\bibliographystyle{plain}/\\bibliographystyle{unsrt}/' relatorio.tex
```

Confirme com `git diff` que apenas a linha esperada mudou, recompile (`make clean && make all`) para conferir que a ordem das referências na bibliografia mudou, depois faça commit e push.

Checkpoint: `git diff HEAD~1` deve mostrar uma alteração de uma única linha em `relatorio.tex`.

Entrega

Envie o link do seu repositório no GitHub para `marco.dias@unifesp.br`. Verifique antes de entregar que:

- ☐ O repositório é público (ou o professor foi adicionado como colaborador).
- ☐ `git log -oneline` mostra o histórico completo pedido nas Partes 1–5 (pelo menos 7 commits distintos).
- ☐ O `.pdf` gerado **não** está versionado (deve estar no `.gitignore`).
- ☐ A partir de um clone limpo, o comando abaixo funciona sem nenhuma intervenção manual:

```
git clone <sua-url>
cd F056
make clean && make all
```

Dica final

Se em algum momento você “perder o controle” do repositório (commits confusos, branch bagunçado), lembre-se do slide “Git: Desfazendo”: `git commit -amend` corrige o último commit, e `git checkout <arquivo>` descarta alterações não commitadas — mas use com cuidado, pois esse último comando é **destrutivo e irreversível**.